

ReelsFlow - Documentação de Engenharia e Especificação de Frontend

v1.0

Este documento estabelece de forma detalhada os padrões arquiteturais, o design system e as especificações de componentes de frontend para a implementação da interface do ReelsFlow. Ele deve ser utilizado como engenharia de contexto para ferramentas de IA focadas em geração de código.

1. Diretrizes de Identidade Visual (Aether Tech Aesthetic)

A interface deve herdar o design escuro premium com profundidade infinita observado nos sistemas corporativos da marca. O layout evita o preto puro (#000000), adotando variações de azul-meia-noite profundo combinadas com desfoques de fundo (backdrop-blur), bordas translúcidas e acentos de luz neon.

1.1. Paleta de Cores (Tailwind Config)

```
const tokens = {
  background: 'bg-[#030712]', // Fundo mestre da aplicação
  surface: 'bg-[#0b1329]/40', // Superfície dos cards Bento Grid
  surfaceMuted: 'bg-[#090f1c]/80', // Sub-cards ou estados desativados
  border: 'border-[#1e293b]/50', // Bordas padrão finas
  borderGlow: 'border-cyan-500/30', // Bordas de foco com brilho
  accent: {
    primary: '#0070f3', // Azul corporativo
    cyan: '#00e5ff', // Cyan/Neon para status e CTA primário
    purple: '#7928ca', // Roxo gradiente para seções especiais
    emerald: '#10b981' // Verde para balanço positivo de créditos
  }
}
```

1.2. Tipografia e Efeitos Visuais

- **Fontes:** Família principal Sans-serif geométrica (ex: Inter ou Sora), com pesos pesados (font-extrabold ou font-bold) para títulos e letter-spacing levemente contraído (tracking-tight).
- **Glow Effect (Efeito Brilho):** Utilização de drop-shadow e box-shadow com opacidade reduzida em elementos interativos e textos principais para simular o visual neon tecnológico (ex: shadow-[0_0_20px_rgba(0,229,255,0.15)]).
- **Cards Transparentes:** Todos os containers do Bento Grid devem usar efeito de vidro fosco (backdrop-blur-md border border-slate-800/60).

2. Arquitetura do Projeto e Estrutura de Pastas

A aplicação será construída sobre o framework Next.js utilizando a App Router estruturada sob TypeScript estrito e Tailwind CSS.

```
src/
├─ app/                                # Roteamento baseado em arquivos (App Router)
│  ├─ (auth)/                          # Grupo de rotas autenticadas (Login/Cadastro)
│  ├─ (dashboard)/                    # Grupo do painel administrativo
│  │  ├─ dashboard/                  # Tela principal (Bento Grid)
│  │  ├─ billing/                   # Controle de créditos e Stripe
│  │  └─ settings/                  # Configuração de tokens da Meta
│  └─ layout.tsx                     # Layout mestre com Providers
├─ components/                        # Componentes atômicos e estruturais
│  ├─ ui/                            # Componentes base (Botões, Inputs, Cards da Bento)
│  └─ dashboard/                     # Componentes específicos do Pannel
│     └─ video/                      # Player 9:16 e Editores de Legenda
├─ hooks/                            # Hooks customizados (useVideoJob, useInstagramAuth)
├─ lib/                              # Utilitários (Supabase Client, API Fetchers)
└─ types/                            # Definições de Tipagem TypeScript Globais
```

3. Layouts e Telas do Sistema (Dashboard)

A área logada principal é composta por uma barra de navegação flutuante superior e uma malha Bento Grid responsiva e funcional.

3.1. Grid de Módulos (Bento Grid Setup)

A tela inicial divide-se em uma malha de doze colunas organizando os módulos de forma fluida:

1. **Card de Geração Core (cols-12 lg:cols-8):** O módulo central da tela. Contém as abas para inserção de "Texto para Vídeo", "Link de Blog" ou "Áudio Trend". Possui o campo de input proeminente com gradiente ativo e botão principal de ação.
2. **Card Player de Visualização 9:16 (cols-12 lg:cols-4 row-span-2):** Um container vertical rígido simulando as dimensões de um smartphone. Quando o vídeo finaliza a renderização em background via n8n, este componente sai do estado de esqueleto (shimmer) e renderiza o player nativo com as legendas dinâmicas aplicadas.
3. **Card de Créditos e Escala (cols-12 md:cols-4):** Inspirado em configuradores laterais de licenciamento. Mostra graficamente os créditos através de uma barra de progresso linear brilhante e atalhos para upgrade transacional.
4. **Fila de Postagens e Histórico (cols-12 md:cols-8):** Uma tabela de listagem limpa que exhibe vídeos agendados, metadados gerados por IA e o status de sincronização com os servidores da Meta.

4. Matriz de Componentes Reutilizáveis (Tailwind CSS)

Componente	Classes Tailwind e Estilização	Comportamento / Micro-interação
Card Bento	bg-[#0b1329]/30 backdrop-blur-md border border-slate-800/80 rounded-2xl p-6 transition-all duration-300 hover:border-cyan-500/30	Efeito de elevação sutil e transição ao passar o mouse, destacando a borda com brilho ciano.
Botão Primário	bg-gradient-to-r from-[#0070f3] to-[#00e5ff] text-white font-semibold px-6 py-3 rounded-xl shadow-lg hover:brightness-110 active:scale-[0.98] transition-all	Botão de ação principal. Apresenta brilho intrínseco e feedback visual tátil instantâneo ao clique.
Badge de Status	bg-cyan-500/10 border border-cyan-500/20 text-[#00e5ff] text-xs px-2.5 py-1 rounded-full flex items-center gap-1.5	Usado para indicar estados de processamento assíncronos em tempo real (ex: "Renderizando").
Input de Texto	bg-[#050b14] border border-slate-800 focus:border-cyan-500/50 focus:ring-1 focus:ring-cyan-500/30 rounded-xl px-4 py-3 text-slate-200 outline-none transition-all w-full	Campo escuro totalmente integrado à UI, alterando o anel de foco dinamicamente.

5. Gerenciamento de Estados e Comunicação Assíncrona

Considerando que o tempo de renderização do vídeo em background pelas APIs gerenciadas via n8n pode durar de 30 a 120 segundos, a interface do cliente necessita operar com gerenciamento reativo.

5.1. Mecanismo de Polling / Server-Sent Events (SSE)

A tipagem base do gerenciamento dos jobs assíncronos obedece ao seguinte contrato:

```
type JobStatus = 'idle' | 'generating_script' | 'rendering_video' | 'ready' | 'failed';

interface VideoJobState {
  jobId: string | null;
  status: JobStatus;
  progress: number; // Intervalo de 0 a 100
  videoUrl: string | null;
  error: string | null;
}
```

- **Estado Crítico de Carregamento (UI Loading):** O player 9:16 deve ocultar os controles padrão de reprodução de mídia e renderizar uma animação cíclica com gradiente translúcido pulsante, fornecendo informações textuais baseadas no estado real do job informado pela API (ex: "Sincronizando áudio neural...", "Renderizando quadros...").
- **Sincronização de Legendas na Interface:** O componente de revisão do roteiro deve fracionar as strings de texto em estruturas mapeadas para permitir a edição granular pelo cliente antes do envio final para a Meta:

```
interface SubtitleBlock {
  id: string;
  text: string;
  startTime: number; // Mapeado em segundos
  endTime: number; // Mapeado em segundos
}
```

Modificações efetuadas pelo usuário nestes blocos disparam requisições PATCH assíncronas que atualizam os metadados do job na base do Supabase, garantindo integridade de dados na renderização final.